NumPy Notes – Array Creation (Complete Guide)

==========ARRAY CREATION==========

There are 6 general mechanisms for creating NumPy arrays:

1) Conversion from Python structures (lists, tuples) 2) Intrinsic NumPy creation functions (arange, zeros, etc.) 3) Replicating, joining, or mutating arrays 4) Reading arrays from disk (CSV, HDF 5 5) Creating arrays from raw bytes or buffers 6) Using special scientific libraries (SciPy, pandas, OpenCV)

This guide focuses on ndarray creation.

---

1) Converting Python Sequences to NumPy Arrays

Lists use [] and tuples use ().

1D array:

```
import numpy as np

a1D = np.array([1, 2, 3, 4])
```

2D array:

```
a2D = np.array([[1, 2], [3, 4]])
```

3D array:

```
a3D = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

Specifying dtype:

```
np.array([127, 128, 129], dtype=np.int8)
```

Note: int8 range is -128 to 127. Values outside this range cause OverflowError.

Mismatching dtype example:

```
a = np.array([2, 3, 4], dtype=np.uint32)
b = np.array([5, 6, 7], dtype=np.uint32)
c_unsigned32 = a - b

c_signed32 = a - b.astype(np.int32)
```

Same dtype → result same type. Different dtype → NumPy promotes to compatible larger type.

Default behavior: - Integers → `32` or `64`-bit (platform dependent) - Floats → float `64`

---

2) Intrinsic NumPy Array Creation Functions

These are divided into: - `1`D array functions - `2`D special matrix functions - General ndarray functions

---

1D Functions

arange():

```
np.arange(10)
np.arange(2, 10, dtype=float)
np.arange(2, 3, 0.1)
```

Best practice: use integer start, stop, step. Note: stop value usually excluded.

linspace():

```
np.linspace(1., 4., 6)
```

Guarantees number of elements and includes stop value.

2D Matrix Functions

Identity matrix:

```
np.eye(3)
np.eye(3, 5)
```

Diagonal matrix:

```
np.diag([1, 2, 3])
np.diag([1, 2, 3], 1)

A = np.array([[1, 2], [3, 4]])
np.diag(A)
```

Vandermonde matrix:

```
np.vander(np.linspace(0, 2, 5), 2)
np.vander([1, 2, 3, 4], 2)
np.vander((1, 2, 3, 4), 4)
```

Useful in polynomial and least squares models.

General ndarray Functions

zeros():

```
np.zeros((2, 3))
np.zeros((2, 3, 2))
```

Default dtype = float `6` `4`

ones():

```
np.ones((2, 3))
np.ones((2, 3, 2))
```

Random arrays:

```
from numpy.random import default_rng
```

```
default_rng(42).random((2,3))
default_rng(42).random((2,3,2))
```

Seed ensures reproducibility.

indices():

```
np.indices((3,3))
```

Useful for multi-dimensional grid evaluation.

---

3) Replicating, Joining, or Mutating Arrays

View example:

```
a = np.array([1, 2, 3, 4, 5, 6])
b = a[:2]
b += 1
```

This modifies original array.

Copy example:

```
a = np.array([1, 2, 3, 4])
b = a[:2].copy()
b += 1
```

Now original is unchanged.

Joining arrays:

```
A = np.ones((2, 2))
B = np.eye(2, 2)
C = np.zeros((2, 2))
D = np.diag((-3, -4))

np.block([[A, B], [C, D]])
```

Other methods: vstack(), hstack()

---

4) Reading Arrays from Disk

Binary formats: - HDF 5 → h 5 py - FITS → Astropy

CSV example:

```
np.loadtxt('simple.csv', delimiter=',', skiprows=1)
```

Other tools: - numpy.genfromtxt - scipy.io - pandas

---

5) Creating Arrays from Raw Bytes

Use: - np.fromfile() - array.tofile()

Be careful with byte order.

---

6) Using Special Libraries

Libraries that use NumPy arrays: - SciPy - pandas - OpenCV

NumPy is the foundation of scientific computing in Python.